

DATABASE CODING/DEVELOPMENT STANDARDS & BEST PRACTICES



GOA ELECTRONICS LIMITED

A Subsidiary of EDC Ltd. (A Government of Goa Undertaking)

Important: The information contained in this document is for exclusive use of Goa Electronics Limited. The information contained in this document should not be passed on to any third party in part or full.

**Revision History :-**

<i>Version</i>	<i>Date</i>	<i>Prepared By</i>	<i>Approved By</i>	<i>Details of Changes</i>
1	22-Jul-2022	Rashmi Gondlekar		
2	22-Sep-2022	Apurva Naik	Anant Yende (CTO)	
3	12-Jul-2024	Gayetri Tirodkar	Anant Yende (CTO)	Document revised by addition of sections like Data type , DataBase table Standards , Store Procedure wrting Standards ,Code formatting , Query optimization, Transactions ,Exception Handling , Documentation , Access and Security , Data Sharing Standards and Database Best Practice.

**TABLE OF CONTENTS**

Sr. No	Title	Page No
1	Naming Conventions	4
2	Data Types	6
3	Variable Declaration	6
4	Database Table Standards	6
5	Store Procedure Writing Standards	8
6	Code Formating	9
7	Query Optimization	9
8	Transactions	9
9	Exception Handling	9
10	Documentation	10
11	Access and Security	10
12	Data Sharing Standards	10
13	Database Best Practice	10



Introduction

This document outlines the coding standards for database development to ensure consistency, maintainability, and performance. Adhering to these standards helps in creating a robust and efficient database environment.

1 Naming conventions :-

1.1 Database :-

The database name should be chosen in such a way that it is clearly able to tell the purpose for which its meant for.

Examples: GoaServices, Bhumija, eDistrict , RERA etc.

1.2 Tables :-

Each new table created should be named in following format .

i) Master or Transaction Tables :-

Format :- <client>_<projectname><tablename>_<tbl/master>

Note:- <projectname> is optional for first project of client

Examples:

Client	:- Director of Mines & Geology (DMG)
Project Name	:- Bhumija (bhum)
Data towards	:- Wallet transaction (wallet)
Transaction/Master	:- tbl/Master
Considering above case table name will be :- dmng_bhumwallet_tbl/dmg_bhumunit_master	

ii) Audit Tables :-

Audit table is mandatory for main transaction tables. Audit table towards the table will be preceded with 'aud' and all the columns of transaction table naming sequence.

Format :- aud_<projectname>_<tablename>_<tbl>

Example : Consider above table audit table format will be as below

Client	:- Director of Mines & Geology (DMG)
Project Name	:- Bhumija (bhum)
Data towards	:- Wallet audit (wallet)
Transaction/Master	:- tbl/Master
Considering above case table name will be :- aud_bhumwallet_tbl	

1.3 Columns :-

Column names to be same as table name except the "tbl"/"master" to be replaced by the column name. Add proper column descriptions which will clearly describe the purpose of the column. Use descriptive names that clearly indicate the column's purpose (e.g., `dmng\_bhumwallet\_id`).



Example: For dmgs_bhumwallet_tbl, column gross amount will be defined as below,
dmgs_bhumwallet_grossAmt

1.4 Indexes :-

Name indexes with a prefix that indicates their type, followed by the table and column names.
Use `idx\_` prefix followed by a descriptive name

Example: `idx\_pwdbill\_lastname` for an index on the `lastname` column of the `bill` table).

1.5 Primary Keys :-

Use `id`/`key` as the primary key for all tables and name it accordingly

Example: `pwd\_bill\_id` (in case integer) or `pwd\_bill\_key` (in case varchar) for the `bill` table)..

1.6 Foreign Keys :-

Name foreign keys with a prefix `fk\_`, followed by the table and column names.

Example : `fk\_pwd\_bill\_id`

1.7 Stored Procedures :-

Use `usp\_` prefix followed by a descriptive name,

Format :- usp_[<projectName >]<Action><table/logical instance : save/delete/populate/populategrid>

Example :- To save data, usp_pwdbilldetail_save
To pouplate data, usp_pwdbilldetail_populate

1.8 Views :-

Use `uvw\_` prefix followed by a descriptive name,

Format :- uvw_<module/table>_<functionality>

Example :- uvw_pwdbill_view

1.9 Sequence :-

Use `seq\_` prefix followed by a descriptive name,

Format :- seq_<module/table>_<idtype>

Example :- seq_pwd_billid

1.10 Materialized View :-

Use `mvw\_` prefix followed by a descriptive name,

Format :- mvw_<module>_populate

Example :- mvw_pwdbill_populate



2. Data Types :-

2.1 Use consistent data types across similar columns in different tables.

(e.g., use `INT` for all ID columns).

2.2 Choose the most appropriate data type for each column.

(e.g., `VARCHAR` for strings, `INT` for integers, `DATE` for dates).

2.3 Standard Field lengths :-

Field	Field Length
All Names	v(150)
Master Name column	v(150)
encAadhar	v(max)
Email id	v(160)
Remarks	v(150) or as per client requirement

2.4 Default values for Parameter / Variables :-

Datatype	Default value
String	" (blank)
Int	0
Float	0.00
Date	01-Jan-1900
Datetime	01-Jan-1900
Bytea/image	null

3. Variable Declaration :-

3.1 Wherever language requires using language specific keywords to be used same to be followed.

Example :- MSSQL requires variables/ parameters to have @ character e.g. @p_name

3.2 Each input parameter of the stored procedure or function should be named in following format

Example :- PostgreSQL --> p_name character varying PostgreSQL

MSSQL --> @p_name varchar(25)

3.4 Each declared variable of the stored procedure should be named in the following format

Variable should be declared along with datatype and valid datasize.

Format :- v_<variable name><datatype><datasize>

Example :- i) PostgreSQL --> v_name character varying(150)

v_srno integer

v_activeflag char

ii) MSSQL --> @v_srno int

@v_name v(100)

4. Database Table Standards :-

Each table should clearly state entry and update/delete conditions and which columns can be updated in update mode and delete mode. Use singular nouns for table names (e.g., `wallet` instead of `wallets`).



- 4.1** Each table must have primary key defined.
- 4.2** The database tables should be in 3NF for OLTP systems.
- 4.3** Make sure that record traceability is available.
- 4.4** Add column level constraints wherever possible as per business rules.
- 4.5** Make sure that column description is added for all columns of the table.
- 4.6** Use realistic field lengths and data types.
- 4.7** Each table must contain following 4 columns by default:
 - ✓ adm_sys_date DATETIME NOT NULL DEFAULT(timestamp without time zone)
 - ✓ adm_off_key VARCHAR(5) NOT NULL
 - ✓ adm_user_key VARCHAR (25) NOT NULL
 - ✓ adm_sys_host VARCHAR (50) NOT NULL

adm_sys_date : Is the date when entry is inserted in table and should be saved as current_timestamp/getdate.
This date cannot be taken as input parameter and must be local value to the server.

adm_off_key : Is the office key of logged in user saved through session.

adm_user_key : Is the user key of logged in user saved through session.

adm_sys_host : Is the IP address of logged in user saved through session.

- 4.8** Audit table will have all the columns of the table which it has used to audit. Audit tables are to capture data changes that have occurred on a database, usually including the information on who made the change, which objects were affected by it and when it was made. Audit table is the replica of main table

- ✓ aud_sys_date DATETIME NOT NULL DEFAULT current_timestamp --> to define changed on date
- ✓ aud_off_key VARCHAR(5) NOT NULL
- ✓ aud_user_key VARCHAR (25) NOT NULL --> to define changed by which user
- ✓ aud_sys_host VARCHAR (50) NOT NULL --> to define changed from which IP

- 4.9** Each table should maintain the status of the record using following column,

Format :- <proj name>_<table name>_activeYn VARCHAR(1) NOT NULL DEFAULT('Y')

Y: When record is active.

N:when inactive or deleted record.

Active YN : Each table will have activeyn flag column. When any new record is inserted in table status of active yn flag will be "Y". If any record is to be deleted from the table, active yn flag towards that record is updated to "N". We have only soft deletion.

- 4.10** Besides above mentioned columns, each table must consist of following columns.

- ✓ <proj name>_<table name>_delReason VARCHAR(100) NOT NULL DEFAULT("")
- ✓ <proj name>_<table name>_delDate DATETIME NOT NULL DEFAULT('01-Jan-1900')
- ✓ <proj name>_<table name>_delUser VARCHAR(25) NOT NULL DEFAULT("")
- ✓ <proj name>_<table name>_delHost VARCHAR(25) NOT NULL DEFAULT("")

When any record is deleted i.e (status of active yn flag towards that entry is updated to 'N') above 4 columns also need to be updated against that deleted record.

delReason : Is the deletion reason to be entered when deleting any record

deldate : is the date time when the record is being deleted

deldate : is the date time when the record is being deleted

delhost : IP of the system from where the record is deleted

**4.11 Constraints :-**

- 4.11.1 Integrity** :- Use constraints to enforce data integrity (e.g., 'NOT NULL', 'UNIQUE', 'CHECK' constraints).
- 4.11.2 Foreign Key** :- Define foreign key constraints to maintain referential integrity between tables.
- 4.11.3 Defaults** :- Use default values for columns where applicable to ensure data consistency.

5. Store Procedure Writing Standards :-

- 5.1** The code of stored procedures (SP) should be clear and concise, following the best practices to write store Procedures. Use comments, indentation, and formatting to make code easy to read and understand. Use consistent and standard syntax, keywords, and operators.
- 5.2** Follow SP naming convention as mentioned in standard 1.7.
- 5.3** The list of parameters to be mentioned first parameter always should be primary key. (e.g. p_applId/p_appkey) followed by other parameters.
- 5.4** Whenever possible avoid making repetitive calls to same table repeatedly.
- 5.5** Ensure the last 4 parameters passed to save SP are,
 - p_desg :- This will be the designation of the user. If designation not available then 0.
 - p_user :- This will be the user id of the user. if user id not available then it will be 'NA'
 - p_office :- This will be the user's office. if office is not available then it will be 'NA'
 - p_host :- This will be the IP address of the system from where transaction is done.
- 5.6** The parameters passed to SP should be as per point 3 of this document.
- 5.7** Type of SP's with return parameter
 - 5.7.1** Save SP will always return character varying.
 - 5.7.2** Populate and Populate grid SP will always return refcursor for PostgreSQL and cursor for MSSQL.
- 5.8** Ensure that SP's are only having specific function/operation. Write smaller modular SP's.
- 5.9** Include comments to describe the purpose, parameters, and return values.
- 5.10** Mentioned author, creation date, description and Jira ticket id. In case existing SP from production is modified mention revised by, revision date, and revision description.

Example :-

AUTHOR NAME : Name of Author
CREATION DATE : Date when SP is executed
Description : Description about SP i.e referred in which module and purpose of it.
REVISION DATE :
REVISED BY :
REVISION DESCRIPTION :

- 5.11** Each SP need to be reviewed by engineer allocated by DataBase Verticle Lead for specific project.
- 5.12** Variable name should start with "v_". Three variables to be defined by default, v_rtn, v_errorYn, v_errorMsg.
- 5.13** Initialize local variables with default values.

Example :- v_rtn:=", v_erroryn := 'N'; v_errormsg := "";



5.14 While updating the existing entry ensure the update has where condition on primary key that is applnId.

5.15 The acknowledgement no (ack_no) and application id/key (appln_id) should not be updated in update statement. Only column allowed for update can be updated in update statement.

6. Code Formatting :-

- 6.1** Indent code to improve readability .
- 6.2** Use single-line comment markers(--)
- 6.3** Reserve multi-line comments (/*.. */) for blocking out sections of code.
- 6.4** Use round brackets to increase readability WHERE (color='red' AND (size = 1 OR size = 2))
- 6.5** Use BEGIN..END blocks.
- 6.6** Use one blank line to separate code sections.
- 6.7** Format JOIN operations using indents .

7. Query Optimization :-

- 7.1 Execution Time** :- Query execution time cannot exceed 2 seconds.
- 7.2 SELECT Statements** :- Only select the columns you need. Don't use `SELECT \*`.
- 7.3 Joins** :- Use joins appropriately and avoid unnecessary complex joins. Do joins on primary key preferably.
- 7.4 Subqueries** :- Use subqueries judiciously. Where possible, prefer joins over subqueries for better performance.
- 7.5 No Functions in WHERE Clauses** :- Don't use functions in WHERE clauses as they can prevent the use of indexes.
- 7.6 Primary Keys** :- Ensure each table has a primary key.
- 7.8 Foreign Keys** :- Create indexes on foreign key columns to improve join performance.
- 7.9 Selective Columns** :- Create indexes on columns that are frequently used in WHERE clauses, ORDER BY clauses, and JOIN conditions.
- 7.10 No Over Indexing** :- Avoid creating too many indexes on a table, as this can degrade write performance. Create maximum three indexes per table.
- 7.11 Multiple execution of same query** :- Avoid querying same data multiple time. Fetch data in variable and then use same variable by calling it multiple times.

8. Transactions :-

- 8.1 Automicity** :- Ensure that each transaction is atomic, meaning that it either fully completes or fully fails.
- 8.2 Scope** :- Keep transactions as short as possible to minimize locking and improve performance.
- 8.3 Error Handling** :- Implement proper error handling and rollback mechanisms within transactions.
- 8.4 Data Validation** :- if same page displays certain options based on certain status in database then while saving such option always validate at database level.

9. Exception Handling :-

- 9.1 Validation** :- Validate data before inserting or updating records to prevent errors.
- 9.2 Error Logging** :- Implement error logging mechanisms to capture and log database errors.



9.3 Graceful Degradation :- Ensure that the application can handle database errors gracefully and provide meaningful error messages.

10. Documentation :-

10.1 Schema Documentation :- Maintain up-to-date documentation of the database schema, including tables, columns, indexes, and relationship with proper versions in google sheet.

10.2 Inline Comments :- Use comments within SQL scripts to explain complex logic and decisions

10.3 Change Logs :- Maintain a change log to document all changes made to the database schema and configurations.

11. Access and Security :-

11.1 Access Control :- Implement strict access control policies to limit who can read, write, and modify data.

11.2 Encryption :- Specific encryption for specific fields is used to protect sensitive data both in transit and at rest.

11.3 Security :- i) Ensure that store procedures, functions, views and triggers do not expose sensitive data like Aadhar No, PAN No, EPIC No, Passport No, Account No, Mobile No, Telephone No, Email Id.
ii) No storage of debit or credit card details into any of the tables.
iii) Aadhar no cannot be stored in any of the table without approval from CTO.

11.4 Database Users and User Privileges :-

The default database user in the database to be disabled. For each project a new database user to be created. The one with limited privilege and one with the administrator privilege. Database users and there privileges should be as follows ,

User	SELECT	INSERT	UPDATE	DELETE	TRUNCATE	REFERENCE	TRIGGER	RULE
DB Admin	☒	☒	☒	☒	☒	☒	☒	☒
Appln User	☒	☒	☒	☐	☐	☒	☒	☐
View User	☒	☐	☐	☐	☐	☒	☒	☐

12. Data Sharing Standards :-

12.1 Aadhar number not to be shared with anyone. It can be shared only after approval from System-Architect/CTO in encrypted format.

12.2 Configuration details not to be shared without approval.

12.3 Data to be only shared with PM and not with anyone else.

12.4 Personal data like Aadhar No, Mobile No, PAN No, EPIC No, Passport No and Bank Account No, as per Data Protect Act, cannot be shared without System Architect/ CTO approval.

13. Database Best Practice :-

13.1 Growth Analysis :- Based on field lengths envisaged compute the database growth analysis based on anticipated transactions.



- 13.2 Data Flow** :- Check data flow viz-a-viz data structure.
- 13.3 Changes** :- Audit the changes.
- 13.4 Flowchart** :- If unsure about the code to be written always draw a flowchart and then proceed to write SP/Trigger/Function/Cursor etc.
- 13.5 Group by** :- Use Group By query precisely. Avoid using multiple column in a group by.
- 13.6 SP Length** :- Ensure each store procedure has maximum 50 lines of code.
- 13.7 Avoid Distinct and Top** :- Avoid using distinct and Top keywords in the queries.
- 13.8 Special operator in Data** :- Never store # or : or / or | or any other operator separated data in a single field.
- 13.9 Parameterization** :- Avoid SQL injection by using parameterized queries instead of dynamic SQL.
- 13.10 Limiting Records** :- Use pagination or SQL limits (Top/Limit) to control data load, especially in reports or large dataset.
- 13.11 Effective Indexing** :- Index column used in where clauses, order by and join predicates.
- 13.12 Avoid Redundancy** :- Do not create indexes that are subsets of existing more comprehensive indexes.
- 13.14 Store Procedures Performance** :- Optimize procedure by avoiding cursors and minimizing loops.
- 13.15 Repeated Queries** :- Avoid querying same table repeatedly for fetching separate columns.
- 13.16 Minimum Privileges** :- Grant users the least privileges which are necessary for their role.